

5. a) Suppose P: $E \rightarrow TE'$

$E' \rightarrow + TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow * FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

Calculate First(), Follow() and STACK implementation of $id*id+id$ using the productions P.

b) P: $S \rightarrow iEtS \mid iEtSeS \mid a$

$E \rightarrow b$

Factorize the stated productions.

c) Define type conversion with an example. State the Type Conversion semantic rules for coercion from integer to real using the following production P.

P: $E \rightarrow num$

$E \rightarrow num . num$

$E \rightarrow id$

$E \rightarrow E1 \text{ op } E2$

6. a) How ϵ -productions are handled in bottom-up parsing?

b) What is semantic analysis in a compiler? Explain its role in the compilation process.

c) Distinguish between syntax analysis and semantic analysis.

d) What is type checking? Explain the difference between static type checking and dynamic type checking.

7. a) Distinguish between top-down parsing (LL) and bottom-up parsing (LR).

b) Given the grammar,

$S \rightarrow AA$

$A \rightarrow aA \mid b$

i) Construct the LR(0) parsing table.

ii) Use the table to parse input string $aabb$

8. a) A syntax-directed definition for declaring identifiers is:

Productions	Semantic Actions
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow int$	$T.type := integer$
$T \rightarrow real$	$T.type := real$
$L \rightarrow L1, id$	$L1.in := L.in,$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

i) Draw an annotated parse tree for the sentence: $real\ id_1, id_2, id_3$

ii) Draw the dependency graph for the parse tree in (i).

b) Distinguish between NFA and DFA in the context of lexical analysis.

c) What is the purpose of a symbol table in a compiler? What are the key attributes stored in a symbol table entry? Explain with an example.



Jashore University of Science and Technology
Department of Computer Science and Engineering

Semester Final Examination-2025
 3rd Year 2nd Semester, Session: 2021-2022

Course Code: CSE-3203

Course Title: Compiler Design and Automata Theory

Time: 3.00 hours

Marks: 72

[N.B. Answer any 6 set questions from the following 8 set questions. The figures in the margin indicate full marks.]

1. a) Is nondeterminism more powerful than determinism in finite automata? Justify your answer. 3
- b) Let R and S be two regular expressions. Then is the following statement true? 4

$$((R^*)^*)^* = R^*$$

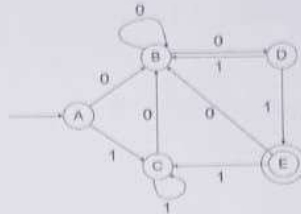
Justify your answer.

- c) Construct an ϵ -NFA equivalent to the regular expression 5

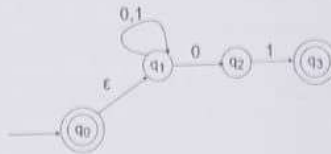
$$(a+b)^*ab$$

Clearly show all ϵ -moves.

2. a) What is a dead state in a deterministic finite automaton (DFA)? Explain with an example. 2
- b) Minimize the following DFA with the table filling method. 7



- c) Convert the following NFA to DFA. 3



3. a) Using pumping lemma prove that the following language: 4

$$L = \{0^n 10^n \mid n \geq 1\}$$

 Is not regular 3+3
- b) Using the grammar 3+3

$$S \rightarrow aSb \mid ab$$

 derive the string **aabb** using:
 i) Leftmost derivation
 ii) Rightmost derivation 2
- c) What is meant by left recursion in a grammar? 2

4. a) Define: Token, Pattern, Lexeme. Identify the tokens and find out the total number of tokens from the following program: 4

```

int main() {
    int a = 17, b = 21, c = 34;
    float d = a * c * 1.5 + b / 2.0;
    printf("Result: %f\n", d);
    return 0;
}
    
```

- b) Define an ambiguous grammar with example. 2
- c) Explain the elimination of left recursion and left factoring in a grammar. Why are these steps necessary for top-down parsing? 6